

Guía de Instalación flutter y Explicación del código

Guía de Configuración de Flutter en Visual Studio Code

Esta guía explica cómo configurar Flutter en Visual Studio Code para desarrollar y ejecutar el código del proyecto. Incluye los pasos básicos para instalar el SDK de Flutter, configurar Visual Studio Code y ejecutar un proyecto.

1. Instalación del Flutter SDK

El primer paso para configurar Flutter es instalar el Flutter SDK. Para eso, se debe ingresar a la página oficial de Flutter (flutter.dev/docs/get-started/install) y descarga el SDK correspondiente a tu sistema operativo. Una vez descargado, se extrae el archivo en una ubicación conveniente en tu computadora, como `C:\flutter` en Windows o una carpeta equivalente en macOS o Linux.

Después de extraer el SDK, se debe agregar la carpeta `flutter/bin` a la variable de entorno `PATH`. Esto permite que el sistema reconozca los comandos de Flutter desde la terminal. Para verificar que todo esté correctamente configurado, abre una terminal y ejecuta el comando `flutter doctor`. Este comando realiza un análisis del entorno de desarrollo e identifica cualquier problema pendiente, como la falta de un emulador o configuraciones adicionales.

2. Configuración de Visual Studio Code

Una vez que Flutter esté instalado, el siguiente paso es configurar Visual Studio Code para trabajar con Flutter. Para lo anterior, se debe abrir Visual Studio Code y acceder a la parte de extensiones haciendo clic en el icono de extensiones en la barra lateral izquierda. Se debe buscar e instalar dos

extensiones clave: **Flutter** y **Dart**. La extensión de Flutter proporciona herramientas específicas para desarrollar aplicaciones con Flutter, mientras que la extensión de Dart incluye soporte para el lenguaje Dart, que es el lenguaje base de Flutter. Una vez instaladas estas extensiones, se debe reiniciar Visual Studio Code para que se apliquen los cambios.

3. Configuración de un Proyecto Flutter

Con Visual Studio Code configurado, se puede crear un primer proyecto Flutter. Para ello, puede abrir Visual Studio Code y presionar **Ctrl+Shift+P** para acceder al menú de comandos, una vez en estado ahí, se escribe **Flutter: New Project** y selecciona esta opción. Se le pedirá que introduzca un nombre para el proyecto y que se seleccione una carpeta donde guardarlo. Una vez creado el proyecto, este se abrirá automáticamente en Visual Studio Code, listo para ser editado y ejecutado.

4. Ejecución de la Aplicación

Antes de ejecutar tu proyecto, asegúrate de tener un dispositivo disponible. Puedes conectar un dispositivo físico con la opción de **Depuración USB** activada o iniciar un emulador desde Android Studio. Visual Studio Code detectará automáticamente los dispositivos conectados, y podrás seleccionarlos desde la barra inferior del editor.

Para ejecutar la aplicación, simplemente presiona **F5** o haz clic en el botón de ejecución en la barra superior de Visual Studio Code. Flutter compilará el proyecto y lo enviará al dispositivo seleccionado. Si todo está configurado correctamente, tu aplicación Flutter se ejecutará en el dispositivo, mostrando la interfaz y funcionalidad predeterminadas del proyecto.

Nota Adicional

Para una mejor comprensión y mayor claridad sobre los pasos antes mencionados, se recomienda seguir la guía en video disponible en el canal de YouTube *munoncode*, titulada *Como Instalar Flutter en Windows (Paso a Paso 2024)*. Este video explica detalladamente el proceso de instalación que se siguió.

El video está disponible en el siguiente enlace: <https://www.youtube.com/watch?v=BTub0BvfEUE>.

Apéndice: Explicación del Código.

Descripción General del Código

Este código fue el utilizado para realizar la aplicación del proyecto . Integra funcionalidades como mapas, gráficos, almacenamiento local y sincronización con Firebase. La aplicación está diseñada en varias pestañas, lo que facilita la organización del código. La pantalla principal, representada por la clase `HomePage`, organiza el acceso a diferentes secciones mediante una barra de navegación inferior, permitiendo al usuario interactuar con las distintas características.

Pantalla Principal (`HomePage`)

La clase `HomePage` actúa como contenedor para las demás vistas de la aplicación. Utiliza un índice (`_currentIndex`) para gestionar qué pantalla se muestra en un momento dado, dependiendo de la selección del usuario en la barra de navegación. Las secciones disponibles incluyen gráficos, mapas, integración con Firebase y un perfil de usuario. Esta pantalla también maneja el almacenamiento local de datos en archivos como `registro_datos.txt` y `matriz_datos.txt`.

Gráficos de Temperatura y Humedad (`ChartsPage`)

La pantalla `ChartsPage` se encarga de mostrar gráficos dinámicos basados en datos obtenidos en tiempo real desde Firebase. Los datos, como temperatura y humedad, se procesan y visualizan utilizando la librería `fl_chart`, que permite renderizar gráficos lineales de forma clara. Los eventos de Firebase son manejados mediante un `StreamSubscription`, lo que asegura que los gráficos se actualicen automáticamente cuando hay cambios en la base de datos. Además, esta pantalla incluye funcionalidad para reiniciar los gráficos, eliminando datos antiguos.

Mapa Interactivo (`MapScreen`)

La clase `MapScreen` proporciona una interfaz para visualizar mapas interactivos utilizando la librería `flutter_map`. El mapa muestra la ubicación actual del usuario, obtenida mediante la librería `Geolocator`, y traza rutas en tiempo real basadas en el movimiento del usuario. La vista incluye un botón para reiniciar el trazado de rutas, lo que resulta útil si el usuario desea comenzar un nuevo registro de su ubicación.

Mapa 3D desde Firebase (FirebasePage)

La pantalla `FirebasePage` procesa datos de sensores obtenidos desde Firebase, como distancias medidas por un sistema de sensores ultrasónicos. Estos datos se organizan en una matriz 4x4, que representa alturas relativas al piso. La aplicación utiliza la librería `flutter_cube` para renderizar un mapa 3D, donde los cubos cambian de color según la altura detectada. Este enfoque visual permite al usuario interpretar fácilmente los datos de los sensores y obtener una representación del entorno.

Gestión de Datos y Perfil (ProfilePage)

La `ProfilePage` está diseñada para gestionar los datos del usuario y las configuraciones de la aplicación. Muestra una matriz con datos recolectados por sensores y permite iniciar el guardado continuo de estos datos en un archivo local. También incluye la funcionalidad de exportar los datos almacenados a la carpeta `Downloads`, facilitando su acceso fuera de la aplicación. Esta pantalla sirve como una herramienta para supervisar y administrar información clave recolectada por la aplicación.

Gestión de la Matriz de Datos (MatrixProvider)

El `MatrixProvider` es una clase encargada de gestionar el estado global de la matriz de datos de sensores. Implementa el patrón de estado global usando la librería `Provider`, lo que permite que los datos sean accesibles y actualizables desde cualquier parte de la aplicación. Cada vez que los datos de la matriz cambian, la clase calcula automáticamente el promedio de las alturas medidas y notifica a las pantallas conectadas para que se actualicen en consecuencia.

Servicios y Funciones Adicionales (`utils.dart` y `LocationService`)

La función `getCurrentLocation` utiliza el `LocationService` para obtener la ubicación actual del usuario. Este servicio se asegura de que los permisos necesarios estén habilitados antes de intentar obtener las coordenadas. En caso de errores, como la falta de permisos, se notifica al usuario mediante mensajes en la interfaz. Además, la función `convertMatrixToText` convierte los datos de la matriz en texto, lo que facilita su almacenamiento en archivos y su exportación.

Punto de Entrada de la Aplicación (`main.dart`)

La clase `MyApp` actúa como el punto de entrada de la aplicación. Inicializa `Firebase` y configura el estado global utilizando `Provider`. Esta clase establece `HomePage` como la pantalla principal, conectando todas las funcionalidades y asegurando que los datos y configuraciones estén listos desde el inicio.

Resumen de Pestañas

Cada archivo de la aplicación en resumen.

- `home_page.dart`: Maneja la pantalla principal y la navegación entre secciones.
- `profile_page.dart`: Gestiona la pantalla de perfil y el guardado/exportación de datos.
- `matrix_provider.dart`: Administra los datos de la matriz y su estado global.
- `utils.dart`: Contiene funciones auxiliares, como la conversión de matrices a texto.
- `charts_page.dart`: Visualiza gráficos dinámicos de temperatura y humedad.
- `mapa_page.dart`: Presenta mapas interactivos con trazado de rutas.
- `firebase_page.dart`: Procesa datos de `Firebase` y renderiza mapas 3D.
- `localitation_page.dart`: Obtiene la ubicación del usuario y maneja permisos de ubicación.